

the block in which they are defined. Tags declared outside of any function must be unique.

3. Structure and union member names. These must be unique within the structure or union in which they are defined. There is no restriction on the type or offset of members with the same member name in different structures.

4. Variables, typedefs, functions, and enumeration members. These must be unique within the scope in which they are defined. Externally declared identifiers must be unique among externally declared variables.

Duplicate names are legal for different name spaces regardless of scope rules.

For example:

```
int blue = 73;
{ // open a block
enumcolors { black, red, green, blue, violet, white } c;
    /* enumerator blue = 3 now hides outer declaration
of int blue */
    structcolors { int i, j; }; // ILLEGAL: colors duplicate
tag
        double red = 2;      // ILLEGAL: redefinition of red
    }
    blue = 37;                // back in int blue scope
```

4.1.5 Functions

Functions are central to C programming. Functions are usually defined as subprograms which return a value based on a number of input parameters. Return value of a function can be used in expressions – technically, function call is considered to be an expression like any other. C allows a function to create results other than its return value, referred to as side effects. Often, function return value is not used at all, depending on the side effects. These functions are equivalent to procedures of other programming languages, such as Pascal. C does not distinguish between procedure and function – functions play both roles.

Each program must have a single external function named `main` marking the entry point of the program. Functions are usually declared as prototypes in standard or user-supplied header files, or within program files. Functions have external linkage by default and are normally accessible from any file in the program. This can be restricted by using the static storage class specifier in function declaration.